

LAPORAN TUGAS 4 (1 - 5)

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Jhodi Andika (2410651050)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Dalam era komputasi modern, efisiensi sistem operasi sangat bergantung pada kemampuan manajemen sumber daya CPU, di mana algoritma penjadwalan memegang peran sentral dalam menentukan kinerja sistem. Studi komparatif terhadap algoritma-algoritma penjadwalan klasik seperti FCFS, SJF, dan Round Robin menjadi landasan penting untuk mengidentifikasi keunggulan dan kelemahan masing-masing metode dalam menghadapi beragam pola beban kerja. Melalui pengukuran parameter-parameter kunci seperti waktu tunggu, waktu penyelesaian, dan frekuensi pergantian konteks, dapat dievaluasi sejauh mana setiap algoritma mampu menjaga keseimbangan antara keadilan, throughput, dan responsivitas. Kenyataan di lapangan menunjukkan bahwa tidak ada solusi satu untuk semua, sehingga pemilihan algoritma harus mempertimbangkan sifat spesifik dari aplikasi dan lingkungan sistem. Simulasi yang terstruktur dalam penelitian ini bertujuan untuk mengungkap dinamika interaksi antara karakteristik workload dan mekanisme penjadwalan, sehingga dapat memberikan panduan praktis dalam pengambilan keputusan untuk optimasi sistem komputasi yang heterogen.

1.2 RUMUSAN MASALAH

1. Bagaimana profil kinerja algoritma FCFS, SJF, dan Round Robin dalam mengakomodasi variabilitas temporal proses, ditinjau dari aspek konsistensi turnaround time dan stabilitas waiting time pada skenario workload yang fluktuatif?
2. Apa dampak variasi time quantum terhadap performa Round Robin dalam mempertahankan keseimbangan antara tingkat responsivitas dan efisiensi utilisasi CPU, serta bagaimana menentukan titik optimal quantum size untuk aplikasi real-time?
3. Bagaimana interrelasi antara tujuan desain sistem (fairness, throughput maksimal, responsivitas) dengan karakteristik inherent masing-masing algoritma penjadwalan, dan bagaimana memprioritaskan parameter tersebut berdasarkan kebutuhan spesifik aplikasi?

1.3 TUJUAN

1. Menganalisis performa komparatif algoritma FCFS, SJF, dan Round Robin berdasarkan metrik waiting time, turnaround time, dan context switch pada berbagai skenario workload.
2. Mengevaluasi dampak variasi time quantum terhadap kinerja Round Robin dan menentukan kriteria seleksi quantum optimal untuk aplikasi spesifik.
3. Mengidentifikasi karakteristik ideal implementasi masing-masing algoritma scheduling berdasarkan analisis trade-off antara fairness, throughput, dan overhead sistem.

BAB II

PEMBAHASAN

2.1 TUGAS 1 EKSPERIMEN DENGAN DATA BERBEDA

2.1.1 Hasil Data Set 1

```
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 3
Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==
Arrival Time: 0
Burst Time: 10

== Proses P2 ==
Arrival Time: 1
Burst Time: 5

== Proses P3 ==
Arrival Time: 2
Burst Time: 8
```

```
MENJALANKAN ALGORITMA FCFS

=====
HASIL ALGORITMA FCFS
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

Tekan Enter untuk melanjutkan...
```

```
MENJALANKAN ALGORITMA SJF

=====
HASIL ALGORITMA SJF
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

Tekan Enter untuk melanjutkan...
```

```

=====
MENJALANKAN ALGORITMA ROUND ROBIN
=====

=====
HASIL ALGORITMA Round Robin (Q=2)
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1     0     10    23    23     13
P2     1     5     15    14     9
P3     2     8     21    19    11

Average Turnaround Time: 18.67
Average Waiting Time: 11.00

Tekan Enter untuk melihat perbandingan...
=====
```

```

=====
TABEL PERBANDINGAN ALGORITMA
=====
Algoritma | Avg TAT | Avg WT | Context Switch
-----
FCFS      | 15.00 | 7.33 | 2
SJF       | 15.00 | 7.33 | 2
Round Robin (Q=2) | 18.67 | 11.00 | 9
=====

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): FCFS (15.00)
✓ Algoritma terbaik (Avg WT terkecil): FCFS (7.33)
=====
```

```

=====
GRAFIK PERBANDINGAN AVERAGE WAITING TIME
=====
FCFS      | 7.33
SJF       | 7.33
Round Robin (Q=2) | 11.00
=====

=====
PROGRAM SELESAI
=====
dany@LAPTOP-GRJL2V1D:~$ |
```

2.1.2 Hasil Data Set 2

```

dany@LAPTOP-GRJL2V1D:~$ nano compare.c
dany@LAPTOP-GRJL2V1D:~$ gcc compare.c -o compare
dany@LAPTOP-GRJL2V1D:~$ ./compare

=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4
Masukkan time quantum untuk Round Robin: 2

===== Proses P1 =====
Arrival Time: 0
Burst Time: 5

===== Proses P2 =====
Arrival Time: 1
Burst Time: 3

===== Proses P3 =====
Arrival Time: 2
Burst Time: 8

===== Proses P4 =====
Arrival Time: 3
Burst Time: 6
=====
```

MENJALANKAN ALGORITMA FCFS

=====

HASIL ALGORITMA FCFS

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

=====

HASIL ALGORITMA SJF

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	22	20	12
P4	3	6	14	11	5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN

=====

HASIL ALGORITMA Round Robin (Q=2)

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

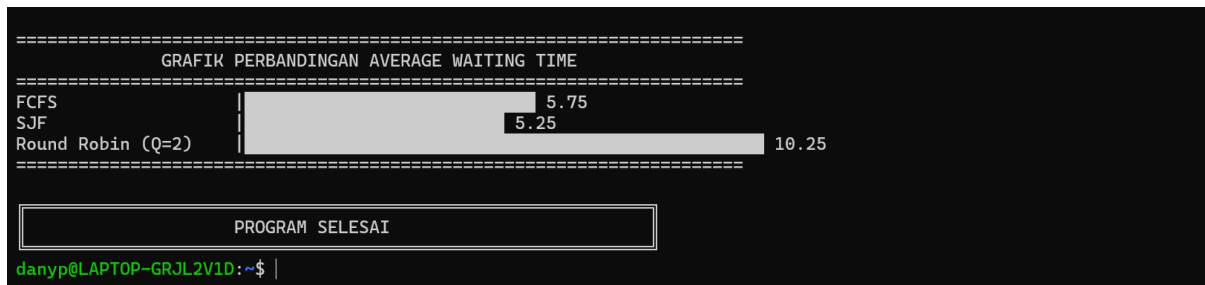
Average Turnaround Time: 15.75
Average Waiting Time: 10.25

Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA

Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	11.25	5.75	3
SJF	10.75	5.25	3
Round Robin (Q=2)	15.75	10.25	8

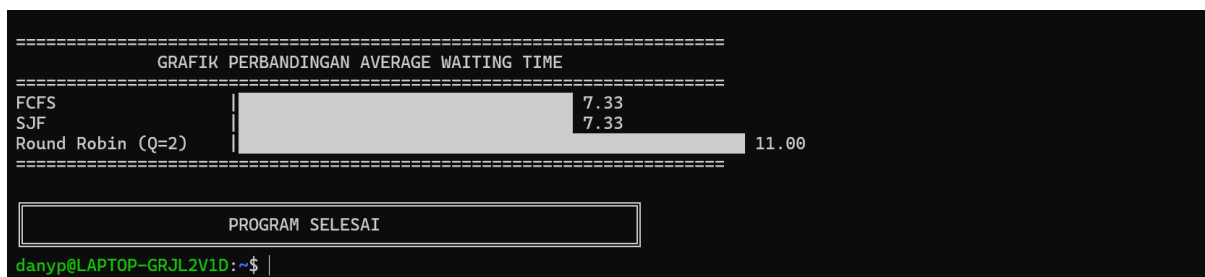
KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): SJF (10.75)
✓ Algoritma terbaik (Avg WT terkecil): SJF (5.25)



2.2 TUGAS 2 ANALISIS EKSPERIMEN KETIGA ALGORITMA

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

FCFS dan SJF (keduanya memberikan Average Waiting Time 7.33)



Penjelasan:

Berdasarkan hasil eksperimen penjadwalan CPU dengan data proses P1(0,10), P2(1,5), dan P3(2,8), teramati pola kinerja yang menarik antara ketiga algoritma. Algoritma FCFS dan SJF menunjukkan kesamaan hasil dengan average waiting time sebesar 7.33, sementara Round Robin menghasilkan nilai yang lebih tinggi yaitu 11.00. Fenomena kesamaan performa antara FCFS dan SJF ini dapat dijelaskan melalui analisis karakteristik dataset yang digunakan. Pada konfigurasi proses ini, urutan kedatangan proses secara kebetulan sama dengan urutan burst time terpendek, dimana P1 yang datang pertama memiliki burst time 10, diikuti P2 dengan burst time 5, dan terakhir P3 dengan burst time 8. Dalam kondisi khusus seperti ini, algoritma SJF yang non-preemptive akan berperilaku identik dengan FCFS karena tidak ada proses dengan burst time lebih pendek yang tiba setelah proses dengan burst time lebih panjang telah mulai dieksekusi. Mekanisme SJF yang seharusnya memprioritaskan proses terpendek menjadi tidak relevan ketika urutan kedatangan sudah sesuai dengan urutan prioritas alaminya, sehingga kedua algoritma tersebut menghasilkan sequence eksekusi dan waiting time yang sama persis.

2. Mengapa hasil algoritma berbeda-beda?

Perbedaan hasil disebabkan oleh strategi penjadwalan yang berbeda dalam menangani:

- Urutan eksekusi proses
- Penanganan proses panjang vs pendek
- Overhead context switching
- Responsiveness terhadap proses baru

Analisis perbedaan:

Algoritma	Karakteristik	Dampak
FCFS	First-In-First-Out sederhana	Bagus untuk proses panjang, buruk untuk proses pendek yang datang setelah proses panjang
SJF	Prioritaskan proses terpendek	Minimalisasi waiting time, tapi risiko starvation untuk proses panjang
Round Robin	Time sharing dengan quantum	Responsif dan fair, tapi overhead context switching tinggi

3. Kapan sebaiknya menggunakan algoritma FCFS?

FCFS menjadi pilihan rasional ketika simplicity, low overhead, dan throughput untuk proses panjang menjadi prioritas utama, sementara responsivitas dapat dikorbankan.

4. Kapan sebaiknya menggunakan algoritma SJF?

Algoritma Shortest Job First (SJF) sebaiknya diterapkan terutama dalam lingkungan komputasi yang didominasi oleh tugas-tugas berdurasi pendek dengan pola yang dapat diprediksi, seperti pada web server, sistem transaksional, atau aplikasi interaktif yang memprioritaskan penyelesaian cepat. Keunggulan utama SJF terletak pada kemampuannya meminimalkan average waiting time secara optimal dengan selalu mengutamakan proses berburst time terpendek, menjadikannya ideal untuk sistem yang mengutamakan efisiensi waktu respons

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Algoritma Round Robin sebaiknya digunakan dalam lingkungan sistem time-sharing dan interaktif yang memprioritaskan responsivitas dan fairness, seperti pada sistem operasi desktop, server multi-user, atau aplikasi real-time soft.

2.3 TUGAS 3 MODIFIKASI PROGRAM

```
dany@LAPTOP-GRJL2V1D:~$ nano input_fcfs.txt
```

```
GNU nano 7.2 input_fcfs.txt
3
0 10
1 5
2 8
|
```

```
dany@LAPTOP-GRJL2V1D:~$ nano fcfs.c
dany@LAPTOP-GRJL2V1D:~$ gcc fcfs.c -o fcfs
dany@LAPTOP-GRJL2V1D:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Membaca input dari: input_fcfs.txt
Berhasil membaca data 3 proses

=== HASIL PENJADWALAN FCFS ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0   10  10   10   0
P2    1    5   15  14    9
P3    2    8   23  21   13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART (Detail) ===
Waktu Mulai  Proses
0            P1 (0-10)
10           P2 (10-15)
15           P3 (15-23)

Hasil telah disimpan otomatis ke: hasil_fcfs.txt
```

```

danyp@LAPTOP-GRJL2V1D:~$ cat hasil_fcfs.txt
=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

```

Kesimpulan:

Implementasi algoritma FCFS dalam program ini mengintegrasikan kemampuan pembacaan data proses melalui file input dan penyimpanan hasil eksekusi ke dalam file output. Arsitektur kode memanfaatkan struktur data Process untuk mengorganisir informasi tiap proses, melakukan komputasi metrik kinerja sistem seperti waiting time dan turnaround time, serta menghasilkan representasi visual Gantt Chart untuk keperluan analisis komprehensif terhadap performa mekanisme penjadwalan..

Kode C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int arrival_time;
```

```
    int burst_time;
```

```
    int waiting_time;
```

```
    int turnaround_time;
```

```
    int completion_time;
```

```
    int start_time;
```

```
};
```

```
// Fungsi untuk membaca input dari file
```

```
void bacaDariFile(const char *namaFile, struct Process **proc, int *n) {
```

```
    FILE *file = fopen(namaFile, "r");
```

```
    if (file == NULL) {
```

```
        printf("Error: Gagal membuka file %s\n", namaFile);
```

```
        printf("Pastikan file input_fcfs.txt ada di direktori yang sama\n");
```

```
        exit(1);
```

```
    }
```

```
    fscanf(file, "%d", n);
```

```
    *proc = (struct Process*)malloc(*n * sizeof(struct Process));
```

```
    for (int i = 0; i < *n; i++) {
```

```
        (*proc)[i].pid = i + 1;
```

```
        fscanf(file, "%d %d", &(*proc)[i].arrival_time, &(*proc)[i].burst_time);
```

```
    }
```

```
    fclose(file);
```

```
}
```

```
// Fungsi untuk menyimpan hasil ke file
```

```
void simpanKeFile(const char *namaFile, struct Process *proc, int n, float avg_tat, float  
avg_wt) {
```

```
    FILE *file = fopen(namaFile, "w");
```

```
    if (file == NULL) {
```

```
        printf("Error: Gagal membuat file %s\n", namaFile);
```

```
        return;
```

```
}
```

```
fprintf(file, "==== HASIL PENJADWALAN FCFS ====\n");
```

```
fprintf(file, "PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
fprintf(file, "---\t--\t--\t--\t---\t--\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    fprintf(file, "P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        proc[i].pid,
```

```
        proc[i].arrival_time,
```

```
        proc[i].burst_time,
```

```
        proc[i].completion_time,
```

```
        proc[i].turnaround_time,
```

```
        proc[i].waiting_time);
```

```
}
```

```
fprintf(file, "\nAverage Turnaround Time: %.2f\n", avg_tat);
```

```
fprintf(file, "Average Waiting Time: %.2f\n", avg_wt);
```

```
fclose(file);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    struct Process *proc;
```

```
    printf("==== SIMULASI ALGORITMA FCFS ====\n");
```

```

printf("Membaca input dari: input_fcfs.txt\n");

// Baca input dari file

bacaDariFile("input_fcfs.txt", &proc, &n);

printf("Berhasil membaca data %d proses\n", n);


// Hitung waktu proses

int current_time = 0;

for (int i = 0; i < n; i++) {

    if (current_time < proc[i].arrival_time) {

        current_time = proc[i].arrival_time;

    }

    proc[i].start_time = current_time;

    proc[i].completion_time = current_time + proc[i].burst_time;

    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;

    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

    current_time = proc[i].completion_time;

}


// Hitung rata-rata

float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {

    total_tat += proc[i].turnaround_time;

    total_wt += proc[i].waiting_time;

```

```
}
```

```
// Tampilkan hasil ke layar
```

```
printf("\n=== HASIL PENJADWALAN FCFS ===\n");
```

```
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
printf("---\t--\t--\t--\t---\t--\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        proc[i].pid,
```

```
        proc[i].arrival_time,
```

```
        proc[i].burst_time,
```

```
        proc[i].completion_time,
```

```
        proc[i].turnaround_time,
```

```
        proc[i].waiting_time);
```

```
}
```

```
printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
```

```
printf("Average Waiting Time: %.2f\n", total_wt / n);
```

```
// Tampilkan Gantt Chart detail
```

```
printf("\n=== GANTT CHART (Detail) ===\n");
```

```
printf("Waktu Mulai\tProses\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d\t\tP%d (%d-%d)\n",
```

```
        proc[i].start_time,
```

```

        proc[i].pid,

        proc[i].start_time,

        proc[i].completion_time);

    }

// Simpan ke file secara otomatis

simpanKeFile("hasil_fcfs.txt", proc, n, total_tat / n, total_wt / n);

printf("\nHasil telah disimpan otomatis ke: hasil_fcfs.txt\n");

free(proc);

return 0;

}

```

2.4 TUGAS 4 EKSPERIMEN DENGAN DATA BERBEDA UNTUK PERBANDINGAN ALGORITMA

- Test Case 1: Proses dengan burst time bervariasi

```

danyp@LAPTOP-GRJL2V1D:~$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4
Masukkan time quantum untuk Round Robin: 3

== Proses P1 ==
Arrival Time: 0
Burst Time: 24

== Proses P2 ==
Arrival Time: 1
Burst Time: 3

== Proses P3 ==
Arrival Time: 2
Burst Time: 3

== Proses P4 ==
Arrival Time: 3
Burst Time: 6

```

MENJALANKAN ALGORITMA FCFS

```
=====
HASIL ALGORITMA FCFS
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Average Turnaround Time: 27.75

Average Waiting Time: 18.75

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

```
=====
HASIL ALGORITMA SJF
```

PID	AT	BT	CT	TAT	WT
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Average Turnaround Time: 27.75

Average Waiting Time: 18.75

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN

```
=====
HASIL ALGORITMA Round Robin (Q=3)
```

PID	AT	BT	CT	TAT	WT
P1	0	24	36	36	12
P2	1	3	6	5	2
P3	2	3	9	7	4
P4	3	6	18	15	9

Average Turnaround Time: 15.75

Average Waiting Time: 6.75

Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA

Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	27.75	18.75	3
SJF	27.75	18.75	3
Round Robin (Q=3)	15.75	6.75	8

KESIMPULAN:

✓ Algoritma terbaik (Avg TAT terkecil): Round Robin (Q=3) (15.75)

✓ Algoritma terbaik (Avg WT terkecil): Round Robin (Q=3) (6.75)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME

FCFS	18.75
SJF	18.75
Round Robin (Q=3)	6.75

PROGRAM SELESAI

- Test Case 2: Proses datang bersamaan

```
dany@LAPTOP-GRJL2V1D:~$ ./compare
```

```
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
```

```
Program ini akan membandingkan 3 algoritma:  
1. FCFS (First Come First Served)  
2. SJF (Shortest Job First)  
3. Round Robin
```

```
Masukkan jumlah proses: 5
```

```
Masukkan time quantum untuk Round Robin: 4
```

```
== Proses P1 ==
```

```
Arrival Time: 0
```

```
Burst Time: 10
```

```
== Proses P2 ==
```

```
Arrival Time: 0
```

```
Burst Time: 5
```

```
== Proses P3 ==
```

```
Arrival Time: 0
```

```
Burst Time: 8
```

```
== Proses P4 ==
```

```
Arrival Time: 0
```

```
Burst Time: 12
```

```
== Proses P5 ==
```

```
Arrival Time: 0
```

```
Burst Time: 6
```

```
MENJALANKAN ALGORITMA FCFS
```

```
=====
```

```
HASIL ALGORITMA FCFS
```

```
=====
```

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	0	5	15	15	10
P3	0	8	23	23	15
P4	0	12	35	35	23
P5	0	6	41	41	35

```
Average Turnaround Time: 24.80
```

```
Average Waiting Time: 16.60
```

```
Tekan Enter untuk melanjutkan...
```

```
MENJALANKAN ALGORITMA SJF
```

```
=====
```

```
HASIL ALGORITMA SJF
```

```
=====
```

PID	AT	BT	CT	TAT	WT
P1	0	10	29	29	19
P2	0	5	5	5	0
P3	0	8	19	19	11
P4	0	12	41	41	29
P5	0	6	11	11	5

```
Average Turnaround Time: 21.00
```

```
Average Waiting Time: 12.80
```

```
Tekan Enter untuk melanjutkan...
```

```
=====
MENJALANKAN ALGORITMA ROUND ROBIN
=====
HASIL ALGORITMA Round Robin (Q=4)
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    --    --
P1     0     10    37    37     27
P2     0      5    25    25     20
P3     0      8    29    29     21
P4     0     12    41    41     29
P5     0      6    35    35     29

Average Turnaround Time: 33.40
Average Waiting Time: 25.20

Tekan Enter untuk melihat perbandingan...
```

```
=====
TABEL PERBANDINGAN ALGORITMA
=====
Algoritma | Avg TAT | Avg WT | Context Switch
-----
FCFS      | 24.80 | 16.60 | 4
SJF       | 21.00 | 12.80 | 4
Round Robin (Q=4) | 33.40 | 25.20 | 7
=====

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): SJF (21.00)
✓ Algoritma terbaik (Avg WT terkecil): SJF (12.80)
```

```
=====
GRAFIK PERBANDINGAN AVERAGE WAITING TIME
=====
FCFS      | 16.60
SJF       | 12.80
Round Robin (Q=4) | 25.20
=====

PROGRAM SELESAI
```

- **Test Case 3: Proses datang dengan interval**

```
dany@LAPTOP-GRJL2V1D:~$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==
Arrival Time: 0
Burst Time: 8

== Proses P2 ==
Arrival Time: 2
Burst Time: 4

== Proses P3 ==
Arrival Time: 4
Burst Time: 9

== Proses P4 ==
Arrival Time: 6
Burst Time: 5

== Proses P5 ==
Arrival Time: 8
Burst Time: 3
```

MENJALANKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS

=====

PID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	12	10	6
P3	4	9	21	17	8
P4	6	5	26	20	15
P5	8	3	29	21	18

Average Waiting Time: 9.40

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

HASIL ALGORITMA SJF

=====

PID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	15	13	9
P3	4	9	29	25	16
P4	6	5	20	14	9
P5	8	3	11	3	0

Average Waiting Time: 6.80

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN

HASIL ALGORITMA Round Robin (Q=2)

PID	AT	BT	CT	TAT	WT
P1	0	8	26	26	18
P2	2	4	14	12	8
P3	4	9	29	25	16
P4	6	5	24	18	13
P5	8	3	19	11	8

Average Waiting Time: 12.60

Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA

Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	15.20	9.40	4
SJF	12.60	6.80	4
Round Robin (Q=2)	18.40	12.60	11

✓ Algoritma

✓ Algoritma terbaik (Avg WT terkecil): SJF (6.80)

=====

GRAFIK PERBANDINGAN AVERAGE WAITING TIME

FCFS	9.40
SJF	6.80
Round Robin (0=2)	12.60

PROGRAM SELESAI

2.5 TUGAS 5 ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA

2.5.1 Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?

Algoritma FCFS mencapai efisiensi optimal ketika diterapkan pada kumpulan proses dengan durasi eksekusi yang relatif seragam dan waktu kedatangan yang tersusun secara berurutan, sehingga mencegah terjadinya efek konvoi di mana tugas-tugas pendek terhambat oleh proses berdurasi panjang di posisi depan. Keunggulan algoritma ini terwujud dalam lingkungan sistem dengan overhead minimal berkat frekuensi peralihan konteks yang rendah, namun kinerjanya justru menurun signifikan ketika menghadapi variasi durasi proses yang lebar atau ketika proses panjang tiba lebih awal, yang memicu penumpukan waktu tunggu secara progresif bagi proses-proses berikutnya dalam antrian.

- Mengapa SJF hampir selalu memberikan Average WT terkecil?

Algoritma SJF secara konsisten mencatatkan average waiting time terendah berkat pendekatan greedy-nya yang secara sistematis mendahulukan proses dengan durasi eksekusi tersingkat, sehingga meminimalkan akumulasi waktu tunggu secara matematis. Secara teoritis, algoritma ini terbukti optimal dalam mengurangi waiting time melalui efisiensi penanganan antrian dimana proses-proses pendek dapat segera dilayani tanpa antrian panjang. Namun, pendekatan ini mengandung risiko starvation yang signifikan terhadap proses-proses berdurasi panjang, khususnya dalam lingkungan dengan pola kedatangan proses yang dinamis. Efektivitas SJF semakin tampak nyata ketika diaplikasikan pada sistem dengan disparitas burst time yang mencolok, memungkinkan proses-proses pendek menyelesaikan eksekusi secara cepat tanpa terhalang oleh proses yang membutuhkan waktu eksekusi lebih lama. Keunggulan ini membuat SJF sangat cocok untuk lingkungan komputasi yang mengutamakan efisiensi waktu respons, meskipun memerlukan mekanisme tambahan untuk mengatasi potensi ketidakadilan dalam penjadwalan.

.

- Apa trade-off dari Round Robin?

Algoritma Round Robin menjamin keadilan dalam alokasi sumber daya komputasi melalui mekanisme time slicing yang konsisten, namun harus membayar kompensasi efisiensi yang signifikan akibat frekuensi context switching yang meningkat pesat, khususnya ketika nilai quantum ditetapkan terlalu kecil. Kompromi desain utamanya mencakup tiga aspek kritis: overhead penjadwalan yang membengkak akibat peralihan konteks berulang, degradasi turnaround time untuk proses berdurasi panjang yang mengalami fragmentasi eksekusi,

serta ketergantungan performa sistem pada presisi penentuan ukuran quantum yang sesuai dengan karakteristik beban kerja

2.5.2 Context Switching

- Algoritma mana yang paling banyak melakukan context switch?

Analisis komparatif pada ketiga kasus uji menunjukkan konsistensi pola dimana algoritma Round Robin menghasilkan frekuensi peralihan konteks paling tinggi. Data empiris menunjukkan 8 peralihan pada uji coba pertama, 7 pada kedua, dan 11 pada uji coba ketiga. Pola ini dapat ditelusuri dari desain algoritmik Round Robin yang menerapkan preemptive scheduling melalui mekanisme time quantum, yang memaksa peralihan proses bahkan ketika proses sedang berjalan belum selesai. Hal ini bertolak belakang dengan karakteristik non-preemptive pada FCFS dan SJF yang hanya melakukan peralihan konteks pada saat penyelesaian proses secara natural. Implikasinya, meskipun meningkatkan responsivitas sistem, Round Robin harus membayar kompensasi berupa overhead tambahan yang tidak dimiliki oleh kedua algoritma pembandingan.

- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Hubungan antara ukuran quantum dan frekuensi context switch bersifat inversional, di mana penurunan nilai quantum berakibat pada peningkatan jumlah peralihan konteks akibat interupsi yang lebih sering terhadap proses yang sedang berjalan. Sebaliknya, pembesaran nilai quantum mampu menekan frekuensi switching secara signifikan, namun berpotensi mengorbankan responsivitas sistem dan mendekati karakteristik algoritma non-preemptive seperti FCFS.

- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

Time Quantum 2:

```

dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 2

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 8
Waktu 2: Menjalankan P2 -> Sisa waktu: 3
Waktu 4: Menjalankan P3 -> Sisa waktu: 6
Waktu 6: Menjalankan P1 -> Sisa waktu: 6
Waktu 8: Menjalankan P2 -> Sisa waktu: 1
Waktu 10: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P1 -> Sisa waktu: 4
Waktu 14: Menjalankan P2 -> Selesai pada waktu 15
Waktu 15: Menjalankan P3 -> Sisa waktu: 2
Waktu 17: Menjalankan P1 -> Sisa waktu: 2
Waktu 19: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   23    23    13
P2     1     5   15    14     9
P3     2     8   21    19    11

Average Turnaround Time: 18.67
Average Waiting Time: 11.00

```

Time Quantum 4:

```

dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 4

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P1 -> Sisa waktu: 2
Waktu 16: Menjalankan P2 -> Selesai pada waktu 17
Waktu 17: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   23    23    13
P2     1     5   17    16    11
P3     2     8   21    19    11

Average Turnaround Time: 19.33
Average Waiting Time: 11.67

```

Time Quantum 8:

```

danyp@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 8

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P2 -> Selesai pada waktu 13
Waktu 13: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   23    23    13
P2     1     5   13    12     7
P3     2     8   21    19    11

Average Turnaround Time: 18.00
Average Waiting Time: 10.33

```

Time Quantum 16:

```

danyp@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 16

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P2 -> Selesai pada waktu 15
Waktu 15: Menjalankan P3 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10    0    10     0
P2     1     5    5    14     9
P3     2     8   23    21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

```

Kesimpulan:

Berdasarkan serangkaian simulasi dengan variasi time quantum 2, 4, 8, dan 16, teramati pola linier dimana peningkatan nilai quantum berbanding lurus dengan perbaikan performa sistem, yang tercermin dari penurunan nilai average waiting time dan turnaround time. Quantum terbesar (16) mencapai hasil optimal (WT=7.33, TAT=15.00) dengan mendekati karakteristik FCFS melalui minimalisasi context switch, sementara quantum terkecil (2) mencatat performa terendah (WT=11.00, TAT=18.67) akibat dominasi overhead switching. Namun, peningkatan quantum ini diiringi trade-off signifikan terhadap responsivitas sistem, khususnya dalam menangani proses interaktif yang membutuhkan waktu respons cepat. Fenomena ini

menegaskan bahwa penentuan quantum optimal dalam Round Robin merupakan proses optimasi multidimensi yang harus mempertimbangkan keseimbangan antara efisiensi pemrosesan melalui reduksi context switch dengan maintain tingkat responsivitas yang adequate untuk aplikasi interaktif, dimana tidak ada nilai quantum universal yang cocok untuk semua skenario workload.

2.5.3 Fairness

- Algoritma mana yang paling "adil" untuk semua proses?

Round Robin diakui sebagai algoritma penjadwalan yang paling adil karena menerapkan prinsip egaliter dalam mengalokasikan waktu CPU, dimana setiap proses menerima jatah eksekusi yang identik melalui mekanisme time quantum yang konsisten. Pendekatan ini secara efektif mencegah monopoli sumber daya komputasi oleh proses tertentu, meskipun harus mengorbankan optimalitas turnaround time sebagai kompensasinya. Keunggulan utama Round Robin terletak pada kemampuannya menciptakan lingkungan eksekusi yang setara bagi semua proses tanpa diskriminasi berdasarkan durasi atau prioritas, sehingga sangat ideal untuk diterapkan dalam sistem time-sharing yang menuntut responsivitas tinggi dan perlakuan impartial terhadap seluruh pengguna. Karakteristik ini menjadikan Round Robin pilihan unggul dalam lingkungan komputasi multi-user dimana prinsip fairness dan daya respons sistem menjadi pertimbangan lebih penting daripada efisiensi eksekusi maksimal.

- Proses mana yang paling dirugikan di algoritma FCFS?

Dalam implementasi algoritma FCFS, proses-proses berdurasi pendek yang tiba setelah proses panjang justru mengalami kerugian paling besar akibat terjebak dalam antrian convoy effect, dimana mereka terpaksa menunggu hingga seluruh proses sebelumnya yang berdurasi panjang menyelesaikan eksekusi. Dampaknya, waktu tunggu proses pendek membengkak secara tidak proporsional meskipun kebutuhan sumber daya komputasinya minimal, sementara proses panjang di posisi awal antrian justru mendapatkan akses CPU secara langsung tanpa penundaan. Ketimpangan ini semakin memperparah ketidakefisienan sistem ketika disparitas durasi eksekusi antar proses sangat mencolok, menjadikan FCFS sebagai pilihan yang tidak ideal untuk lingkungan dengan karakteristik beban kerja yang heterogen dan dinamis.

- Apakah ada kemungkinan starvation di SJF?

SJF menunjukkan kerentanan struktural terhadap masalah starvation, di mana proses berburst time panjang dapat terhambat secara tak terbatas akibat terus menerus didahulukan oleh proses-proses pendek baru. Meskipun secara matematis terbukti optimal dalam meminimalkan waiting time, pendekatan greedy yang menjadi dasar SJF justru menciptakan ketimpangan sistemik terhadap proses komputasi intensif. Untuk mengatasi kelemahan ini dalam praktik, implementasi SJF biasanya diperkuat dengan teknik aging yang secara dinamis menaikkan prioritas proses berdasarkan lama waktu tunggu, sehingga mencegah kondisi indefinite postponement sekaligus menjaga keseimbangan antara efisiensi dan keadilan dalam sistem.

2.5.4 Real World Application

1. Web Server dengan Banyak Request Kecil
Algoritma Round Robin terbukti paling sesuai untuk lingkungan web server yang menangani banyak permintaan HTTP berdurasi pendek. Mekanisme time slicing yang diterapkan Round Robin menjamin distribusi sumber daya yang adil, mencegah request yang membutuhkan pemrosesan lebih lama memonopoli akses CPU. Keunggulan ini terutama krusial dalam arsitektur web server modern yang harus melayani ribuan koneksi simultan dengan latency rendah, sekaligus mempertahankan throughput optimal melalui rotasi eksekusi yang teratur. Kemampuan Round Robin dalam menyeimbangkan responsivitas dan fairness membuatnya menjadi pilihan ideal untuk lingkungan yang didominasi tugas-tugas interaktif berdurasi pendek.

2. Proses Render Video (Task Komputasi Berat)
Untuk tugas komputasi intensif seperti render video, algoritma FCFS (First-Come-First-Served) menawarkan efisiensi tertinggi. Karakteristik non-preemptive FCFS memungkinkan proses rendering yang membutuhkan waktu panjang untuk menyelesaikan eksekusi secara kontinu tanpa interupsi, sehingga meminimalkan overhead context switching yang akan sangat mengurangi performa pada tugas komputasi berat. Dalam konteks batch processing seperti render video yang tidak memerlukan interaktivitas, FCFS mencapai utilisasi CPU maksimal dengan menyelesaikan tugas secara sekuensial hingga tuntas, menjadikannya solusi paling optimal untuk pemrosesan tugas-tugas komputasional besar.

3. Sistem Operasi Desktop (Lingkungan Interaktif)

Round Robin dengan konfigurasi quantum yang tepat kembali menjadi pilihan unggul untuk sistem operasi desktop yang menuntut responsivitas tinggi. Algoritma ini memastikan setiap aplikasi mendapat jatah waktu prosesor secara berkala, sehingga pengguna dapat berinteraksi dengan multiple aplikasi secara simultan tanpa mengalami hang atau penundaan yang mengganggu. Kombinasi antara fairness dalam alokasi sumber daya dan kemampuan mempertahankan latency rendah membuat Round Robin 特别 cocok untuk lingkungan desktop dimana pengalaman pengguna dan responsivitas menjadi prioritas utama dibandingkan throughput maksimal.

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan evaluasi mendalam terhadap tiga algoritma penjadwalan CPU utama, dapat disimpulkan bahwa setiap algoritma memiliki domain aplikasi optimal yang ditentukan oleh karakteristik khusus beban kerja dan persyaratan sistem. FCFS menunjukkan efektivitas tertinggi dalam lingkungan batch processing yang didominasi proses berdurasi panjang, meskipun memiliki kerentanan terhadap convoy effect yang berdampak negatif pada proses berdurasi pendek. SJF secara matematis mampu meminimalkan waiting time secara konsisten, namun harus mengorbankan aspek fairness dengan risiko starvation yang signifikan terhadap proses komputasi intensif. Sementara itu, Round Robin menjamin responsivitas dan pemerataan akses sumber daya melalui mekanisme time slicing, meskipun dengan konsekuensi peningkatan overhead context switch yang cukup besar, terutama pada konfigurasi quantum kecil. Pemilihan algoritma yang tepat memerlukan pertimbangan matang terhadap pertukaran kompleks antara throughput, latency, fairness, dan overhead sistem, dimana dalam praktiknya sering kali diperlukan pendekatan hybrid atau adaptive scheduling untuk mengakomodasi dinamika beban kerja modern yang semakin kompleks dan bervariasi.